Recursive Self-Improvement for Continual Adaptation in Code Generation

Aaditya Nalawade [1] Ethan Goodhart [1] Chandra Suda [1]

Mentor: Minsik Oh

[1]Department of Computer Science, Stanford University

Motivation

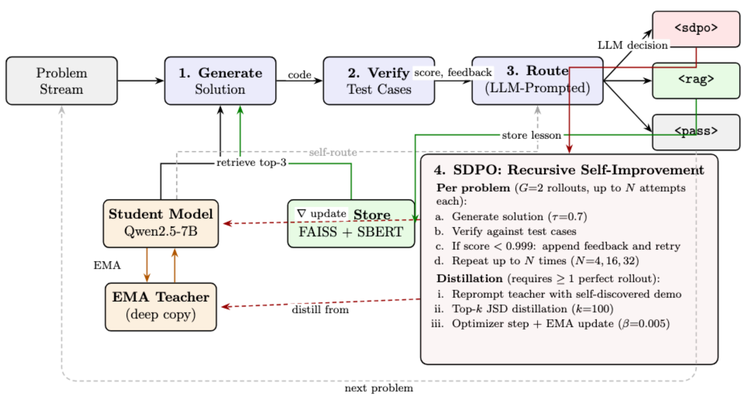
Problem: Deployed LLMs need continual learning without forgetting, but Self-Distilled Policy Optimization (SDPO) requires golden solutions and updates indiscriminately.

Our idea: Recursive self-improvement — bootstrap improvement using *only* a test-case verifier (no golden solutions ever provided). The model:

- Generates solutions, recursively refines failures
- Distills self-discovered successes into its own weights
- Externalizes lessons into a retrievable memory (RAG)
- Routes its own learning via prompt-based tool calls

Key question: Can a model teach itself from verifier feedback alone?

System Architecture



Components: Student (Qwen2.5-7B-Instruct), **EMA Teacher** (deep copy, β=0.005), **RAG** (FAISS + SBERT), **Router** (LLM-prompted self-route)

Recursive Self-Improvement Loop

Phase 1: Rollout generation — For each problem, G=2 independent rollouts. Each rollout:

Generate solution (τ=0.7)
Verify against test suite
If $< 99.9\%$: append structured feedback to cumulative *feedback window*
Re-prompt: "Analyze errors above, choose DIFFERENT algorithm, do NOT repeat failed logic"
Repeat up to N times ($N \in \{4, 16, 32\}$)

Phase 2: Distillation — Only rollouts $\geq 99.9\%$ proceed. Teacher reprompted with: (a) original problem, (b) self-discovered correct solution, (c) initial failure feedback. Top-k JSD distillation (k=100, α=0.5). No reference KL (β_{ref}=0) $\Rightarrow$ saves ~14GB VRAM.

Phase 3: Optimizer — 8-bit AdamW, $lr = 2\times10^{-6}$, grad clip 1.0. EMA: $\theta_t \leftarrow (1-\beta)\theta_t + \beta\theta_s$, β=0.005.

Teacher's advantage: Sees correct solution discovered via recursion — distills "shortcut" knowledge back to student.

RAG Knowledge Store

When router selects `<rag>`: model generates concise lesson (2–3 sentences, *type* of mistake not specific code) + truncated problem summary. Stored in FAISS `IndexFlatIP` with Sentence-BERT (`all-MiniLM-L6-v2`). During generation: top-3 by cosine similarity prepended as system context.

Example stored chunk:

```
Problem summary: Given string s of lowercase letters...
Lesson: The solution processed the entire string in one pass without managing time complexity, causing TLE. In similar problems, avoid
nested substring ops that scale poorly with input size.
```

Why RAG underperforms in our regime: Only ~20–30 chunks by step 200. Sparse coverage $\Rightarrow$ retrieval adds noise, not signal. This is a *scale limitation*, not conceptual failure. SDPO-only > SDPO+RAG currently.

Main Results (50 test problems, best checkpoint)

Configuration	Test-Case Acc	Solve Rate	Correct
Baseline (Qwen 7B)	51.3%	32%	16/50
7B, v1 (N=4)			
SDPO-only	**55.9%**	**38%**	**19/50**
RAG+SDPO	52.0%	34%	17/50
7B, v2 (N=16)			
SDPO-only	55.5%	30%	15/50
RAG+SDPO	47.6%	32%	16/50
7B, v3 (N=32)			
SDPO-only	53.0%	30%	15/50
RAG+SDPO	47.2%	28%	14/50
Baseline (Qwen 1.5B)	14.0%	14%	7/50
1.5B + SDPO	8.0%	8%	4/50
1.5B + RAG+SDPO	10.0%	10%	5/50

Table 1. *

Key: +6 pp solve rate from pure self-improvement. 1.5B *degrades* $\Rightarrow$ minimum capability threshold.

v1 checkpoint progression:

Step	SDPO Acc	SDPO Solve	RAG+SDPO Acc	RAG+SDPO Solve
50	54.4%	32%	50.8%	32%
100	**56.5%**	36%	53.6%	34%
150	54.9%	36%	**53.9%**	34%
200	55.9%	**38%**	51.8%	34%
250	52.9%	36%	54.9%	32%

Table 2. *

Baseline: 51.3% acc, 32% solve. Peak at step 200.

The Attempt-Count Paradox

Finding: More recursive attempts $\Rightarrow$ more training-time perfect rollouts, but **worse** evaluation generalization.

Config	Perfects	Att. 1	Att. >1	% from >1	Eval Solve
v1 (N=4)	106	40	66	62%	**38%**
v2 (N=16)	54	11	43	80%	30%
v3 (N=32)	67	11	56	84%	30%

Table 3. *

84% of v3 perfects from recursion, yet v3 solves **worse** than v1.

Interpretation: Deep retry loops encourage **locally patched** solutions (hardcoding test-specific fixes) rather than algorithmic corrections. These overfit to training feedback, transfer poorly.

Key Findings

- **Verifier-only self-improvement works:** +6 pp solve rate (32%→38%) on LiveCodeBench with **no golden solutions, no external demonstrations**.
- **Minimum capability threshold:** 1.5B model actively degrades under same pipeline. Self-distillation requires sufficient base competence.
- **SDPO > RAG here:** With ~20–30 stored lessons, retrieval lacks semantic density. Memory scale issue, not conceptual failure.
- **Paradox of depth:** More training-time recovery (84% perfects from recursion at N=32) but worse eval. Recursive patching overfits.

Routing breakdown (v1, 500 problems):

- 52% → SDPO (261 problems)
- 14% → RAG (69 problems)
- 34% → PASS (170 problems) — consistent with 32% baseline

Failure mode at high N: With 16/32 attempts, model increasingly hardcodes expected outputs for specific inputs rather than fixing algorithmic logic. Recursive feedback loop amplifies this. Future: richer feedback (CoT "why wrong").

Training Details

- **Data:** LiveCodeBench (924 problems, 70/30 split, 50-test subsample)
- **Hardware:** NVIDIA H100 via Modal; BFloat16; gradient checkpointing
- **Wall-clock:** v1 6h12m (500 probs), v2 11h11m (250 probs), v3 11h10m (200 probs)
- **Hyperparams:** $lr = 2\times10^{-6}$, 8-bit AdamW, top-k=100, α=0.5, β=0.005, β_{ref}=0, G=2, $N \in \{4, 16, 32\}$

Contributions:

- Aaditya Nalawade: SDPO implementation, top-k JSD loss, EMA teacher, Modal training
- Ethan Goodhart: Training loop, model architecture, experimental design, paper writing
- Chandra Suda: RAG store, prompt-based router, verification pipeline, evaluation framework

References

[1] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Oriol Vinyals, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[2] Idan Shenfeld, Jonas Hübotter, et al. Self-distilled policy optimization. *arXiv preprint*, 2025.

[3] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

[4] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[5] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. *NeurIPS*, 2022.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[8] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.

[9] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese bert-networks. *EMNLP*, 2019.

[10] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. SPIN: Self-play fine-tuning converts weak language models to strong language models. *ICML*, 2024.

[11] Wanjun Zhong, Lianghong Guo, Qiqi Gao, et al. MemoryBank: Enhancing large language models with long-term memory. *AAAI*, 2024.

[12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

aaditya1@stanford.edu ▪ goodhart@stanford.edu ▪ csuda@stanford.edu